

PROPUESTA DE VIDEOJUEGO — CRÓNICAS DEL TIEMPO (DISEÑO PARA C++/QT)

1. Sinopsis general

Crónicas del Tiempo es un videojuego sencillo diseñado para ser implementado en C++ con Qt (QGraphicsView/QGraphicsScene), compuesto por tres niveles, cada uno inspirado en un acontecimiento histórico distinto. El objetivo del jugador es superar los retos particulares de cada escena ajustándose a diferentes dinámicas, físicas y tipos de cámara. El juego busca ser corto, agradable a la vista y con dificultad progresiva pero justa. La entrega actual corresponde únicamente a la propuesta de diseño (sin programación).

2. Objetivos de aprendizaje del curso

- Aplicar principios de Programación Orientada a Objetos (POO) en el modelado de niveles, entidades y físicas.
- Planear el uso de memoria dinámica segura (punteros inteligentes) para la futura implementación.
- Diseñar la interfaz gráfica (GUI) con Qt y un bucle de juego basado en QTimer (planificado).
- Definir y parametrizar al menos tres modelos físicos (ecuaciones) distribuidos entre los niveles.
- Incorporar un agente autónomo con percepción, razonamiento, acción y aprendizaje simples.

3. Visión creativa y alcance

La propuesta abarca tres niveles breves e independientes, cada uno con identidad visual y mecánicas diferenciadas. Se evitarán mecánicas de plataformas; en cambio, se usarán esquemas de desplazamiento lateral sin saltos entre plataformas, vista cenital con scroll y control de caída/aterrizaje vertical. Al menos un nivel estará controlado por tiempo.

4. Niveles propuestos (temática histórica 2025-2)

Nivel 1 — El Mensajero de Maratón (490 a. C.)

- Vista: lateral fija (no plataformero).
- Contexto histórico (sinopsis): inspirado en la legendaria carrera del mensajero griego tras la batalla de Maratón.
- Dinámicas principales:
 - Gestión de resistencia; el jugador regula el ritmo para evitar agotarse.
 - Esquiva de proyectiles (flechas) con perturbaciones por viento.
 - HUD con temporizador: el reto es llegar antes de que el tiempo expire.
- Retos y objetivos: llegar a la meta con vida y dentro del límite temporal.
- Físicas (parametrizables):
 - 1) Movimiento amortiguado del corredor: $x(t+\Delta)=x(t)+v_x\Delta$; $v_x(t+\Delta)=v_x(t)+a_x\Delta-c\cdot v_x\Delta$
 - 2) Trayectoria de flecha oscilatoria: $x(t)=x_0+A\cdot\sin(\omega t+\phi)$; $y(t)=y_0+v_y\cdot t$
 - 3) Perturbación de viento: $\Delta x_v = k\cdot\sin(2\pi f\cdot t)$
- Funcionamiento general: cámara fija lateral, QTimer (futuro) para actualización, indicadores de vida/tiempo/estamina.
- Este nivel cumple el requisito de control por tiempo.

Nivel 2 — Caravana en la Ruta de la Seda (siglo XIV)

- Vista: cenital con scroll limitado (la cámara sigue al jugador; mapa con rutas).
- Contexto histórico (sinopsis): travesía mercantil entre puestos de comercio en Asia central.
- Dinámicas principales:
 - Terrenos con diferentes coeficientes de arrastre (arena, camino, roca).
 - Tormentas que generan campos de viento no uniformes.
 - Agente autónomo (bandido) que patrulla y aprende rutas frecuentadas por el jugador.
- Retos y objetivos: alcanzar sucesivos puestos sin ser interceptado y optimizando trayectorias.
- Físicas (parametrizables):
 - 1) Movimiento con arrastre: $v(t+\Delta)=v(t)+a\Delta-k\cdot v(t)\Delta$
 - 2) Campo de viento: $w(x,y,t)=\langle a\cdot\sin(\omega t+\alpha x), b\cdot\sin(\omega t+\beta y)\rangle$
 - 3) Partículas de arena con pequeñas parábolas para efecto visual y retroalimentación.
- Funcionamiento general: navegación libre, decisiones tácticas frente a clima y rutas; HUD de distancia/alerta de avistamiento.

Nivel 3 — Alunizaje 1969

- Vista: vertical tipo 'lander' (no plataformas).
- Contexto histórico (sinopsis): descenso controlado de un módulo lunar inspirado en la misión Apolo 11.

- Dinámicas principales:
 - Gestión de combustible vs. empuje.
 - Control de orientación y velocidad de descenso.
 - Zona de aterrizaje con tolerancias estrictas.
- Retos y objetivos: posar el módulo dentro de umbrales de $|v_y|$, $|v_x|$ y $|\theta|$ con combustible limitado.
- Físicas (parametrizables):
 - 1) Gravedad + empuje: $a = \langle (T/m) \cdot \cos\theta, (T/m) \cdot \sin\theta - g \rangle$
 - 2) Rotación con inercia: $\theta(t+\Delta) = \theta(t) + \omega\Delta$; $\omega(t+\Delta) = \omega(t) + \tau/I \cdot \Delta$
 - 3) Colisión parcialmente elástica (rebotar suave si se excede levemente la velocidad).
- Funcionamiento general: control fino de motores, feedback visual/auditivo, medidor de combustible y vector velocidad.

5. Vistas, Interacciones y Físicas de Cada Nivel (Momento II – Punto 1)

5.1. Nivel 1 — El Mensajero de Maratón

5.1.1. Vista del Nivel

El Nivel 1 utiliza una **vista lateral fija 2D**, sin desplazamiento vertical y sin mecánicas de plataforma tradicionales. El jugador se desplaza únicamente en el eje horizontal, y la cámara permanece estática, mostrando una porción del terreno con horizonte ligeramente inclinado para reforzar la sensación de carrera.

El HUD muestra:

- Barra de estamina,
- Barra de vida,
- Cronómetro de tiempo restante,
- Indicador de distancia a la meta.

La composición visual incluye: cielo, montañas, camino de tierra y elementos históricos como estandartes y ruinas, todo diseñado para transmitir velocidad y tensión.

5.1.2. Interacciones del Nivel

- **Jugador ↔ Terreno:** la fricción del camino afecta la velocidad real del mensajero.
- **Jugador ↔ Estamina:** correr más rápido reduce estamina; si se agota, la velocidad cae abruptamente.
- **Jugador ↔ proyectiles (flechas):** colisiones directas reducen vida.
- **Jugador ↔ Viento:** pequeñas perturbaciones horizontales afectan la trayectoria del jugador.
- **Jugador ↔ Objetivo:** llegar a la meta antes de que el temporizador llegue a cero genera victoria.

5.1.3. Físicas Utilizadas

1. Movimiento amortiguado del corredor

$$x(t + \Delta t) = x(t) + v_x \Delta t \quad v_x(t + \Delta t) = v_x(t) + a_x \Delta t - c \cdot v_x \Delta t$$

El término c modela la resistencia física del corredor.

2. Trayectoria oscilatoria de flechas con viento

$$x(t) = x_0 + A \sin(\omega t + \phi) y(t) = y_0 + v_y t$$

Las flechas no siguen trayectorias parabólicas típicas; la oscilación horizontal simula vibración y viento.

3. Perturbación del viento sobre el jugador

$$\Delta x_v = k \cdot \sin(2\pi f t)$$

5.1.4. Sketch del Nivel (Descripción)

El sketch muestra el terreno horizontal, el jugador en el lado izquierdo, proyectiles viniendo desde la derecha y elementos del entorno estáticos. La zona objetivo se ubica al fondo con un marcador claro.

5.2. Nivel 2 — Caravana en la Ruta de la Seda

5.2.1. Vista del Nivel

La vista es cenital con ligero desplazamiento (scroll). La cámara sigue al jugador dentro de un mapa grande, con caminos, zonas de arena, áreas rocosas y rutas comerciales.

El HUD contiene indicadores de:

- dirección del viento,
- nivel de arrastre según el terreno,
- radar de proximidad del bandido.

5.2.2. Interacciones del Nivel

- Jugador ↔ Terreno: cada tipo de suelo modifica el coeficiente de arrastre.
- Jugador ↔ Viento: el viento aplica fuerzas vectoriales variables.
- Jugador ↔ Bandido: si entra en su campo de visión, el agente cambia de estado y persigue.
- Jugador ↔ Puestos comerciales: puntos seguros para llegar al siguiente tramo del nivel.
- Jugador ↔ Tormentas: zonas con visibilidad reducida y arrastre impredecible.

5.2.3. Físicas Utilizadas

1. Movimiento con arrastre

$$v(t + \Delta) = v(t) + a\Delta - k \cdot v(t)\Delta$$

2. Campo de viento dependiente de posición y tiempo

$$w(x, y, t) = \langle a \sin(\omega t + \alpha x), b \sin(\omega t + \beta y) \rangle$$

3. Partículas de arena (efecto visual):

Pequeñas parábolas con baja gravedad simulando turbulencia ligera.

5.2.4. Sketch del Nivel (Descripción)

Se representa un mapa con caminos principales, áreas arenosas, regiones rocosas y la ubicación inicial del bandido. El jugador se desplaza por corredores abiertos, con flechas indicando direcciones del viento.

5.3. Nivel 3 — Alunizaje 1969

5.3.1. Vista del Nivel

La vista es **vertical tipo “lander”**, donde el jugador observa el módulo descendiendo desde arriba hacia la superficie lunar. El HUD presenta combustible, velocidad vertical, velocidad horizontal y orientación del módulo.

5.3.2. Interacciones del Nivel

- **Jugador ↔ Gravedad:** la aceleración vertical es constante hacia abajo.
- **Jugador ↔ Motores:** el jugador regula empuje y orientación.
- **Jugador ↔ Superficie:** si el impacto excede tolerancias, se considera un fallo.
- **Jugador ↔ Combustible:** cada impulso reduce la reserva disponible.

5.3.3. Físicas Utilizadas

1. Aceleración por empuje más gravedad

$$a_x = \frac{T}{m} \cos(\theta) a_y = \frac{T}{m} \sin(\theta) - g$$

2. Rotación con inercia

$$\theta(t + \Delta) = \theta(t) + \omega \Delta \quad \omega(t + \Delta) = \omega(t) + \frac{\tau}{I} \Delta$$

3. Colisión parcialmente elástica

Si la velocidad excede ligeramente los umbrales, el módulo rebota con energía reducida.

5.3.4. Sketch del Nivel (Descripción)

El dibujo muestra el módulo en descenso, con flechas vectoriales indicando velocidades y fuerzas, zona de aterrizaje delimitada y barras de combustible y velocidad.

6. Sprites por Nivel (Momento II – Punto 2)

6.1. Sprites del Nivel 1



6.2. Sprites del Nivel 2



6.3. Sprites del Nivel 3



7. Diagrama de Clases UML (Momento II – Punto 3)

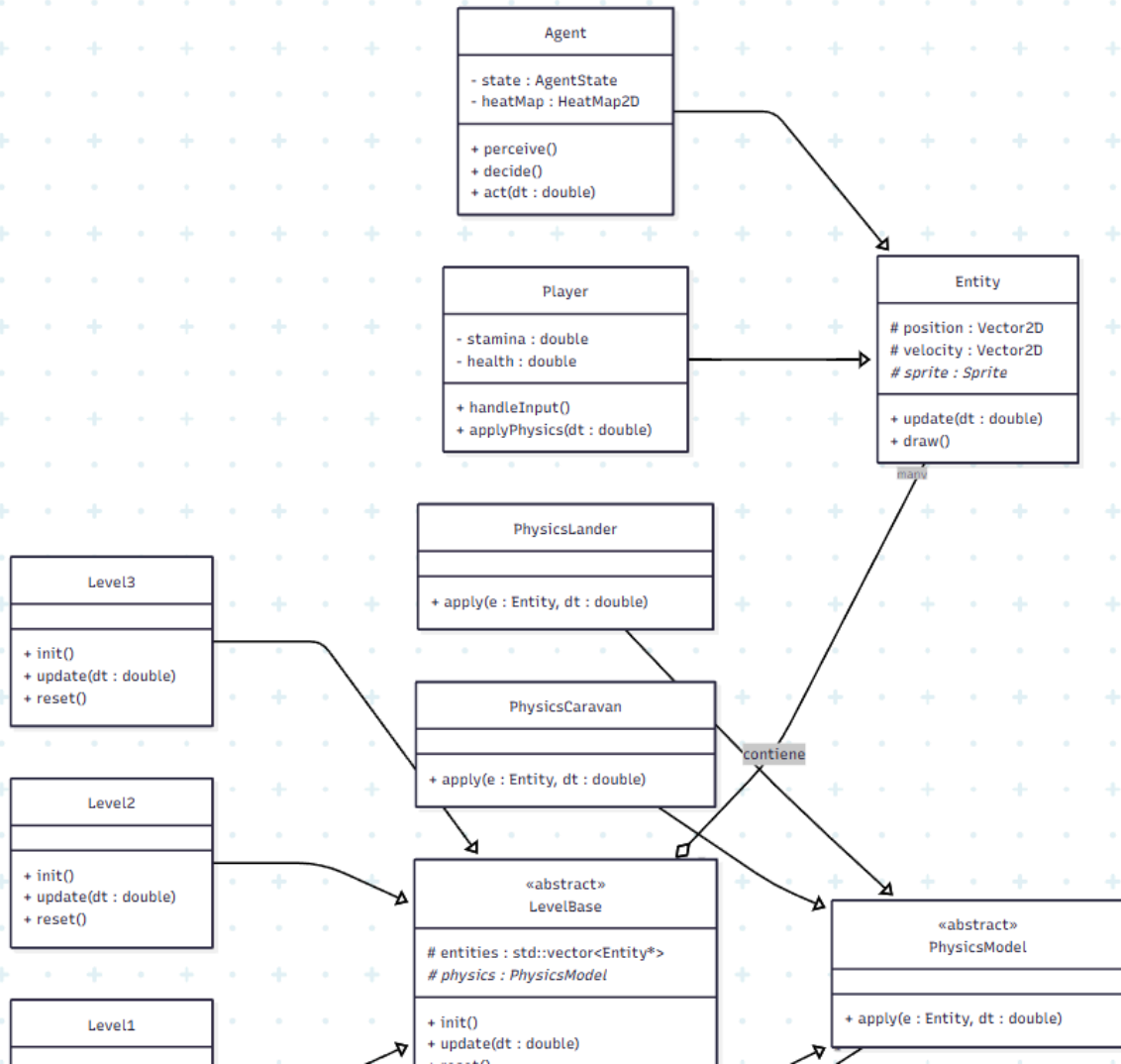
7.1. Descripción general de la arquitectura

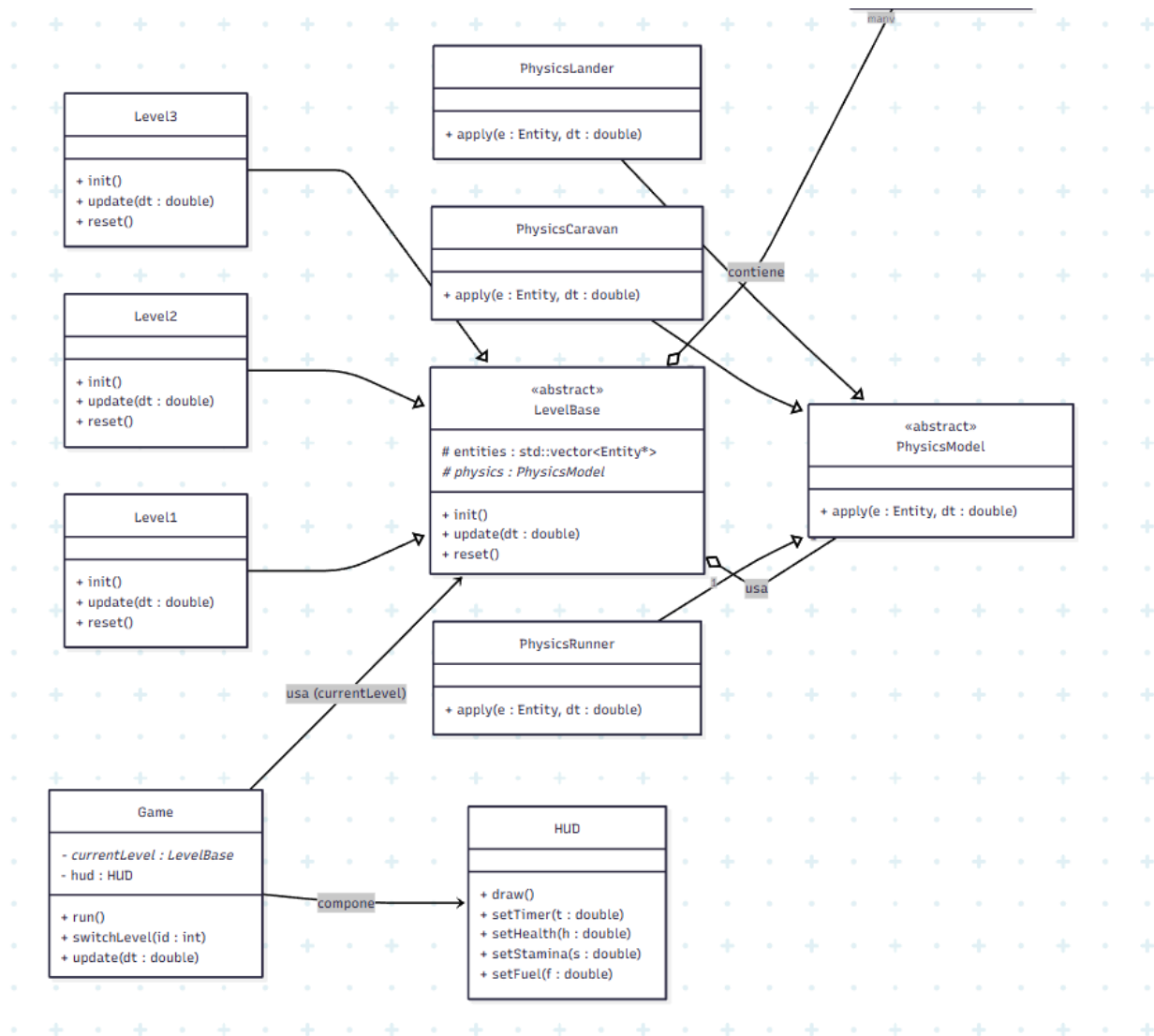
El videojuego sigue un enfoque de **POO modular**. La clase principal Game contiene la escena activa, el HUD y controla la transición entre niveles.

Cada nivel deriva de una clase abstracta LevelBase.

Las entidades (jugador, enemigos, proyectiles, módulo lunar) derivan de Entity.

7.2. Diagrama UML de nivel medio/alto





7.3. Uso de herencia (explicación)

La herencia se utiliza para:

- Definir un marco común de niveles.
- Definir un modelo genérico de entidad que luego es especialización.
- Compartir comportamiento base y permitir polimorfismo en la actualización del juego.

7.4. Contenedores STL empleados y justificación

- `std::vector<Entity*>` → lista de entidades en pantalla.
- `std::map<std::string, double>` → parámetros de físicas configurables.
- `std::queue<Event>` → cola de eventos para el sistema de colisiones.

- std::unique_ptr → gestión de niveles evitando fugas de memoria.

Cada contenedor se selecciona por eficiencia y claridad estructural.

8. Descripción del Agente Inteligente (Momento II – Punto 4)

8.1. Rol del Agente

El agente aparece en el Nivel 2 como un **bandido autónomo** cuya función principal es detectar, perseguir y anticipar el movimiento del jugador, aumentando la tensión y obligando a una planificación de rutas.

8.2. Percepción

- Cono de visión frontal de 60°.
- Detección por ruido con umbral variable.
- Sensores de proximidad activados por distancia.
- Detección de áreas de calor (zonas frecuentadas por el jugador).

8.3. Razonamiento / Máquina de Estados

La lógica se basa en una **FSM (Finite State Machine)** con estados:

1. Patrullar

- o Movimiento cíclico por rutas predefinidas.

2. Perseguir

- o Si ve al jugador, acelera hacia él.

3. Anticipar

- o Si pierde al jugador, extrapola la dirección previa.

4. **Buscar**

- o Explora zonas calientes del mapa.

El agente cambia de estado según condiciones de percepción.

8.4. **Acciones Posibles**

- Moverse en patrones definidos.
- Acelerar y cambiar de dirección.
- Bloquear caminos colocando trampas temporales.
- Reducir velocidad en terrenos de alta fricción.
- Reportar internamente zonas visitadas por el jugador.

8.5. **Aprendizaje**

El agente utiliza una técnica simple de mapa de calor:

- El mapa es una matriz 2D donde cada celda aumenta cuando el jugador pasa sobre ella.
- El agente emplea política **ϵ -greedy** para decidir rutas:
 - o Con probabilidad $1 - \epsilon$ elige la celda más visitada.
 - o Con probabilidad ϵ explora nuevas zonas.
- Esto permite que aprenda rutas frecuentes.

consolidar una visi

El diagrama UML elaborado constituye un avance significativo hacia la estructuración modular del proyecto. La separación clara entre niveles, entidades, sistema de físicas, HUD y agente autónomo facilita el mantenimiento, la extensibilidad y el uso seguro de memoria mediante punteros inteligentes. El diseño plantea un marco flexible en el que cada componente cumple una responsabilidad precisa, evitando acoplamientos innecesarios y fomentando el polimorfismo y la reutilización de código.

El análisis del agente inteligente permitió demostrar la incorporación de técnicas de IA acordes al alcance del curso, integrando percepción, razonamiento mediante máquina de estados, comportamiento adaptativo y un sistema básico de aprendizaje mediante mapa de calor. Este módulo no solo agrega profundidad jugable, sino que introduce al estudiante a principios de inteligencia artificial aplicados en un contexto interactivo realista.

En conjunto, el trabajo desarrollado en esta fase evidencia un avance maduro del proyecto, pasando de una idea creativa a un diseño formal listo para implementación. La documentación generada establece bases sólidas para la programación del videojuego, garantizando claridad conceptual, coherencia técnica y una arquitectura preparada para escalar en complejidad durante fases posteriores.